

Fig. 10. A simple entity-relationship diagram

3. ENTITY-RELATIONSHIP DIAGRAM AND INCLUSION OF SEMANTICS IN DATA DESCRIPTION AND MANIPULATION

3.1 System Analysis Using the Entity-Relationship Diagram

In this section we introduce a diagrammatic technique for exhibiting entities and relationships: the entity-relationship diagram.

Figure 10 illustrates the relationship set PROJECT-WORKER and the entity sets EMPLOYEE and PROJECT using this diagrammatic technique. Each entity set is represented by a rectangular box, and each relationship set is represented by a diamond-shaped box. The fact that the relationship set PROJECT-WORKER is defined on the entity sets EMPLOYEE and PROJECT is represented by the lines connecting the rectangular boxes. The roles of the entities in the relationship are stated.

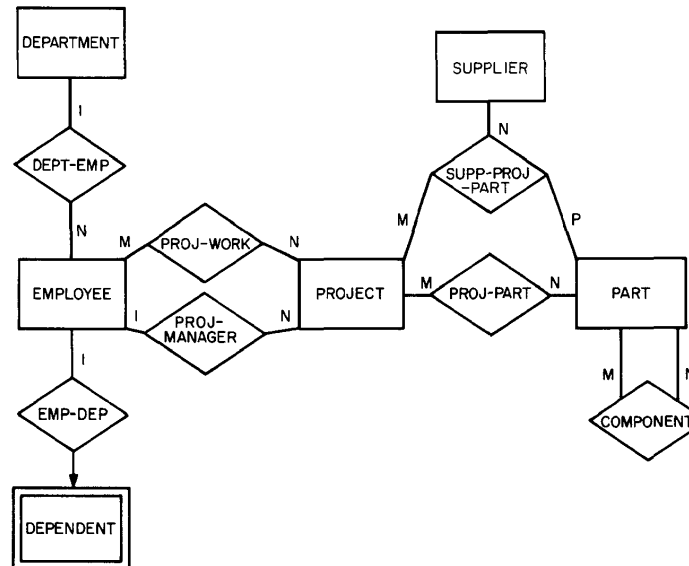


Fig. 11. An entity-relationship diagram for analysis of information in a manufacturing firm

Figure 11 illustrates a more complete diagram of some entity sets and relationship sets which might be of interest to a manufacturing company. DEPARTMENT, EMPLOYEE, DEPENDENT, PROJECT, SUPPLIER, and PART are entity sets. DEPARTMENT-EMPLOYEE, EMPLOYEE-DEPENDENT, PROJECT-WORKER, PROJECT-MANAGER, SUPPLIER-PROJECT-PART, PROJECT-PART, and COMPONENT are relationship sets. The COMPONENT relationship describes what subparts (and quantities) are needed in making superparts. The meaning of the other relationship sets need not be explained.

Several important characteristics about relationships in general can be found in Figure 11:

(1) A relationship set may be defined on more than two entity sets. For example, the SUPPLIER-PROJECT-PART relationship set is defined on three entity sets: SUPPLIER, PROJECT, and PART.

(2) A relationship set may be defined on only one entity set. For example, the relationship set COMPONENT is defined on one entity set, PART.

(3) There may be more than one relationship set defined on given entity sets. For example, the relationship sets PROJECT-WORKER and PROJECT-MANAGER are defined on the entity sets PROJECT and EMPLOYEE.

(4) The diagram can distinguish between $1:n$, $m:n$, and $1:1$ mappings. The relationship set DEPARTMENT-EMPLOYEE is a $1:n$ mapping, that is, one department may have n ($n = 0, 1, 2, \dots$) employees and each employee works for only one department. The relationship set PROJECT-WORKER is an $m:n$ mapping, that is, each project may have zero, one, or more employees assigned to it and each employee may be assigned to zero, one, or more projects. It is also possible to express $1:1$ mappings such as the relationship set MARRIAGE. Information about the number of entities in each entity set which is allowed in a relationship set is indicated by specifying “1”, “ m ”, “ n ” in the diagram. The relational model and the entity set model² do not include this type of information; the network model cannot express a $1:1$ mapping easily.

(5) The diagram can express the *existence dependency* of one entity type on another. For example, the arrow in the relationship set EMPLOYEE-DEPENDENT indicates that existence of an entity in the entity set DEPENDENT depends on the corresponding entity in the entity set EMPLOYEE. That is, if an employee leaves the company, his dependents may no longer be of interest.

Note that the entity set DEPENDENT is shown as a special rectangular box. This indicates that at level 2 the information about entities in this set is organized as a weak entity relation (using the primary key of EMPLOYEE as a part of its primary key).

3.2 An Example of a Database Design and Description

There are four steps in designing a database using the entity-relationship model: (1) identify the entity sets and the relationship sets of interest; (2) identify semantic information in the relationship sets such as whether a certain relationship

² This mapping information is included in DIAM II [24].

set is an $1:n$ mapping; (3) define the value sets and attributes; (4) organize data into entity/relationship relations and decide primary keys.

Let us use the manufacturing company discussed in Section 3.1 as an example. The results of the first two steps of database design are expressed in an entity-relationship diagram as shown in Figure 11. The third step is to define value sets and attributes (see Figures 2 and 3). The fourth step is to decide the primary keys for the entities and the relationships and to organize data as entity/relationship relations. Note that each entity/relationship set in Figure 11 has a corresponding entity/relationship relation. We shall use the names of the entity sets (at level 1) as the names of the corresponding entity/relationship relations (at level 2) as long as no confusion will result.

At the end of the section, we illustrate a schema (data definition) for a small part of the database in the above manufacturing company example (the syntax of the data definition is not important). Note that value sets are defined with specifications of representations and allowable values. For example, values in EMPLOYEE-NO are represented as 4-digit integers and range from 0 to 2000. We then declare three entity relations: EMPLOYEE, PROJECT, and DEPENDENT. The attributes and value sets defined on the entity sets as well as the primary keys are stated. DEPENDENT is a weak entity relation since it uses EMPLOYEE.PK as part of its primary key. We also declare two relationship relations: PROJECT-WORKER and EMPLOYEE-DEPENDENT. The roles and involved entities in the relationships are specified. We use EMPLOYEE.PK to indicate the name of the entity relation (EMPLOYEE) and whatever attribute-value-set pairs are used as the primary keys in that entity relation. The maximum number of entities from an entity set in a relation is stated. For example, PROJECT-WORKER is an $m:n$ mapping. We may specify the values of m and n . We may also specify the minimum number of entities in addition to the maximum number. EMPLOYEE-DEPENDENT is a weak relationship relation since one of the related entity relations, DEPENDENT, is a weak entity relation. Note that the existence dependence of the dependents on the supporter is also stated.

<u>DECLARE</u>	<u>VALUE-SETS</u>	<u>REPRESENTATION</u>	<u>ALLOWABLE-VALUES</u>
	EMPLOYEE-NO	INTEGER (4)	(0,2000)
	FIRST-NAME	CHARACTER (8)	ALL
	LAST-NAME	CHARACTER (10)	ALL
	NO-OF-YEARS	INTEGER (3)	(0,100)
	PROJECT-NO	INTEGER (3)	(1,500)
	PERCENTAGE	FIXED (5.2)	(0,100.00)
<u>DECLARE</u>	<u>REGULAR ENTITY RELATION EMPLOYEE</u>		
	<u>ATTRIBUTE/VALUE-SET:</u>		
	EMPLOYEE-NO/EMPLOYEE-NO		
	NAME/(FIRST-NAME, LAST-NAME)		
	ALTERNATIVE-NAME/(FIRST-NAME,LAST-NAME)		
	AGE/NO-OF-YEARS		
	<u>PRIMARY KEY:</u>		
	EMPLOYEE-NO		

```

DECLARE      REGULAR ENTITY RELATION PROJECT
               ATTRIBUTE/VALUE-SET:
               PROJECT-NO/PROJECT-NO
               PRIMARY KEY:
               PROJECT-NO

DECLARE      REGULAR RELATIONSHIP RELATION PROJECT-WORKER
               ROLE/ENTITY-RELATION.PK/MAX-NO-OF-ENTITIES
               WORKER/EMPLOYEE.PK/m
               PROJECT/PROJECT.PK/n    (m:n mapping)
               ATTRIBUTE/VALUE-SET:
               PERCENTAGE-OF-TIME/PERCENTAGE

DECLARE      WEAK RELATIONSHIP RELATION EMPLOYEE-DEPENDENT
               ROLE/ENTITY-RELATION.PK/MAX-NO-OF-ENTITIES
               SUPPORTER/EMPLOYEE.PK/1
               DEPENDENT/DEPENDENT.PK/n
               EXISTENCE OF DEPENDENT DEPENDS ON
               EXISTENCE OF SUPPORTER

DECLARE      WEAK ENTITY RELATION DEPENDENT
               ATTRIBUTE/VALUE-SET:
               NAME/FIRST-NAME
               AGE/NO-OF-YEARS
               PRIMARY KEY:
               NAME
               EMPLOYEE.PK THROUGH EMPLOYEE-DEPENDENT

```

3.3 Implications on Data Integrity

Some work has been done on data integrity for other models [8, 14, 16, 28]. With explicit concepts of entity and relationship, the entity-relationship model will be useful in understanding and specifying constraints for maintaining data integrity. For example, there are three major kinds of constraints on values:

(1) Constraints on *allowable values* for a value set. This point was discussed in defining the schema in Section 3.2.

(2) Constraints on *permitted values* for a certain attribute. In some cases, not all allowable values in a value set are permitted for some attributes. For example, we may have a restriction of ages of employees to between 20 and 65. That is,

$$\text{AGE}(e) \in (20,65), \text{ where } e \in \text{EMPLOYEE}.$$

Note that we use the level 1 notations to clarify the semantics. Since each entity/relationship set has a corresponding entity/relationship relation, the above expression can be easily translated into level 2 notations.

(3) Constraints on *existing values* in the database. There are two types of constraints:

(i) Constraints between sets of existing values. For example,

$$\{\text{NAME}(e) \mid e \in \text{MALE-PERSON}\} \subseteq \{\text{NAME}(e) \mid e \in \text{PERSON}\}.$$

(ii) Constraints between particular values. For example,

$$\begin{aligned} & \text{TAX}(e) \leq \text{SALARY}(e), e \in \text{EMPLOYEE} \\ & \text{or} \\ & \text{BUDGET}(e_i) = \sum \text{BUDGET}(e_j), \text{ where } e_i \in \text{COMPANY} \\ & \quad e_j \in \text{DEPARTMENT} \\ & \text{and } [e_i, e_j] \in \text{COMPANY-DEPARTMENT}. \end{aligned}$$

3.4 Semantics and Set Operations of Information Retrieval Requests

The semantics of information retrieval requests become very clear if the requests are based on the entity-relationship model of data. For clarity, we first discuss the situation at level 1. Conceptually, the information elements are organized as in Figures 4 and 5 (on Figures 2 and 3). Many information retrieval requests can be considered as a combination of the following basic types of operations:

- (1) Selection of a subset of values from a value set.
- (2) Selection of a subset of entities from an entity set (i.e. selection of certain rows in Figure 4). Entities are selected by stating the values of certain attributes (i.e. subsets of value sets) and/or their relationships with other entities.
- (3) Selection of a subset of relationships from a relationship set (i.e. selection of certain rows in Figure 5). Relationships are selected by stating the values of certain attribute(s) and/or by identifying certain entities in the relationship.
- (4) Selection of a subset of attributes (i.e. selection of columns in Figures 4 and 5).

An information retrieval request like “What are the ages of the employees whose weights are greater than 170 and who are assigned to the project with PROJECT-NO 254?” can be expressed as:

$\{\text{AGE}(e) \mid e \in \text{EMPLOYEE}, \text{WEIGHT}(e) > 170,$
 $[e, e_j] \in \text{PROJECT-WORKER}, e_j \in \text{PROJECT},$
 $\text{PROJECT-NO}(e_j) = 254\};$

or

$\{\text{AGE}(\text{EMPLOYEE}) \mid \text{WEIGHT}(\text{EMPLOYEE}) > 170,$
 $[\text{EMPLOYEE}, \text{PROJECT}] \in \text{PROJECT-WORKER},$
 $\text{PROJECT-NO}(\text{EMPLOYEE}) = 254\}.$

To retrieve information as organized in Figure 6 at level 2, “entities” and “relationships” in (2) and (3) should be replaced by “entity PK” and “relationship PK.” The above information retrieval request can be expressed as:

$\{\text{AGE}(\text{EMPLOYEE.PK}) \mid \text{WEIGHT}(\text{EMPLOYEE.PK}) > 170$
 $(\text{WORKER/EMPLOYEE.PK}, \text{PROJECT/PROJECT.PK}) \in \{\text{PROJECT-WORKER.PK}\},$
 $\text{PROJECT-NO}(\text{PROJECT.PK}) = 254\}.$

To retrieve information as organized in entity/relationship relations (Figures 7, 8, and 9), we can express it in a SEQUEL-like language [6]:

```
SELECT    AGE
FROM      EMPLOYEE
WHERE     WEIGHT > 170
```

Table I. Insertion

level 1	level 2
<i>operation:</i> insert an entity to an entity set	<i>operation:</i> create an entity tuple with a certain entity-PK <i>check:</i> whether PK already exists or is acceptable
<i>operation:</i> insert a relationship in a relationship set <i>check:</i> whether the entities exist	<i>operation:</i> create a relationship tuple with given entity PKs <i>check:</i> whether the entity PKs exist
<i>operation:</i> insert properties of an entity or a relationship <i>check:</i> whether the value is acceptable	<i>operation:</i> insert values in an entity tuple or a relationship tuple <i>check:</i> whether the values are acceptable

AND EMPLOYEE.PK =
 SELECT WORKER/EMPLOYEE.PK
 FROM PROJECT-WORKER
 WHERE PROJECT-NO = 254.

It is possible to retrieve information about entities in two different entity sets without specifying a relationship between them. For example, an information retrieval request like "List the names of employees and ships which have the same

Table II. Updating

level 1	level 2
<i>operation:</i> change the value of an entity attribute	<i>operation:</i> - update a value <i>consequence:</i> - if it is not part of an entity PK, no consequence - if it is part of an entity PK, - change the entity PKs in all related relationship relations - change PKs of other entities which use this value as part of their PKs (for example, DEPENDENTS' PKs use EMPLOYEE'S PK)
<i>operation:</i> change the value of a relationship attribute	<i>operation:</i> update a value (note that a relationship attribute will not be a relationship PK)

Table III. Deletion

level 1	level 2
<i>operation:</i> • delete an entity <i>consequences:</i> • delete any entity whose existence depends on this entity • delete relationships involving this entity • delete all related properties	<i>operation:</i> • delete an entity tuple <i>consequences (applied recursively):</i> • delete any entity tuple whose existence depends on this entity tuple • delete relationship tuples associated with this entity
<i>operation:</i> • delete a relationship <i>consequences:</i> • delete all related properties	<i>operation:</i> • delete a relationship tuple

age'' can be expressed in the level 1 notation as:

$$\{(\text{NAME}(e_i), \text{NAME}(e_j)) \mid e_i \in \text{EMPLOYEE}, e_j \in \text{SHIP}, \text{AGE}(e_i) = \text{AGE}(e_j)\}.$$

We do not further discuss the language syntax here. What we wish to stress is that information requests may be expressed using set notions and set operations [17], and the request semantics are very clear in adopting this point of view.

3.5 Semantics and Rules for Insertion, Deletion, and Updating

It is always a difficult problem to maintain data consistency following insertion, deletion, and updating of data in the database. One of the major reasons is that the semantics and consequences of insertion, deletion, and updating operations usually are not clearly defined; thus it is difficult to find a set of rules which can enforce data consistency. We shall see that this data consistency problem becomes simpler using the entity-relationship model.

In Tables I–III, we discuss the semantics and rules³ for insertion, deletion, and updating at both level 1 and level 2. Level 1 is used to clarify the semantics.

4. ANALYSIS OF OTHER DATA MODELS AND THEIR DERIVATION FROM THE ENTITY-RELATIONSHIP MODEL

4.1 The Relational Model

4.1.1 The Relational View of Data and Ambiguity in Semantics. In the relational model, *relation*, R , is a mathematical relation defined on sets $X_1, X_2, \dots, X_n$:

$$R = \{(x_1, x_2, \dots, x_n) \mid x_1 \in X_1, x_2 \in X_2, \dots, x_n \in X_n\}.$$

The sets $X_1, X_2, \dots, X_n$ are called *domains*, and $(x_1, x_2, \dots, x_n)$ is called a *tuple*. Figure 12 illustrates a relation called EMPLOYEE. The domains in the relation

³Our main purpose is to illustrate the semantics of data manipulation operations. Therefore, these rules may not be complete. Note that the consequence of operations stated in the tables can be performed by the system instead of by the users.

ROLE		LEGAL	LEGAL	ALTERNATIVE	ALTERNATIVE	
DOMAIN	EMPLOYEE- NO	FIRST- NAME	LAST- NAME	FIRST- NAME	LAST- NAME	NO-OF- YEARS
TUPLE	2566	PETER	JONES	SAM	JONES	25
	3378	MARY	CHEN	BARB	CHEN	23

Fig. 12. Relation EMPLOYEE

are EMPLOYEE-NO, FIRST-NAME, LAST-NAME, FIRST-NAME, LAST-NAME, NO-OF-YEAR. The ordering of rows and columns in the relation has no significance. To avoid ambiguity of columns with the same domain in a relation, domain names are qualified by *roles* (to distinguish the role of the domain in the relation). For example, in relation EMPLOYEE, domains FIRST-NAME and LAST-NAME may be qualified by roles LEGAL or ALTERNATIVE. An *attribute name* in the relational model is a domain name concatenated with a role name [10]. Comparing Figure 12 with Figure 7, we can see that “domains” are basically equivalent to value sets. Although “role” or “attribute” in the relational model seems to serve the same purpose as “attribute” in the entity-relationship model, the semantics of these terms are different. The “role” or “attribute” in the relational model is mainly used to distinguish domains with the same name in the same relation, while “attribute” in the entity-relationship model is a function which maps from an entity (or relationship) set into value set(s).

Using relational operators in the relational model may cause semantic ambiguities. For example, the join of the relation EMPLOYEE with the relation EMPLOYEE-PROJECT (Figure 13) on domain EMPLOYEE-NO produces the

PROJECT-NO	EMPLOYEE-NO
7	2566
3	2566
7	3378

Fig. 13. Relation EMPLOYEE-PROJECT

		LEGAL	LEGAL	ALTERNATIVE	ALTERNATIVE	
PROJECT- NO	EMPLOYEE- NO	FIRST- NAME	LAST- NAME	FIRST- NAME	LAST- NAME	NO-OF- YEARS
7	2566	PETER	JONES	SAM	JONES	25
3	2566	PETER	JONES	SAM	JONES	25
7	3378	MARY	CHEN	BARB	CHEN	23

Fig. 14. Relation EMPLOYEE-PROJECT' as a "join" of relations EMPLOYEE and EMPLOYEE-PROJECT

relation EMPLOYEE-PROJECT' (Figure 14). But what is the meaning of a join between the relation EMPLOYEE with the relation SHIP on the domain NO-OF-YEARS (Figure 15)? The problem is that the same domain name may have different semantics in different relations (note that a role is intended to distinguish domains in a given relation, not in all relations). If the domain NO-OF-YEAR of the relation EMPLOYEE is not allowed to be compared with the domain NO-OF-YEAR of the relation SHIP, different domain names have to be declared. But if such a comparison is acceptable, can the database system warn the user?

In the entity-relationship model, the semantics of data are much more apparent. For example, one column in the example stated above contains the values of AGE of EMPLOYEE and the other column contains the values of AGE of SHIP. If this semantic information is exposed to the user, he may operate more cautiously (refer to the sample information retrieval requests stated in Section 3.4). Since the database system contains the semantic information, it should be able to warn the user of the potential problems for a proposed "join-like" operation.

4.1.2 Semantics of Functional Dependencies Among Data. In the relational model, "attribute" B of a relation is *functionally dependent* on "attribute" A of the same relation if each value of A has no more than one value of B associated with it in the relation. Semantics of functional dependencies among data become clear

SHIP-NO	NAME	NO-OF-YEARS
037	MISSOURI	25
056	VIRGINIA	10

Fig. 15. Relation SHIP

in the entity-relationship model. Basically, there are two major types of functional dependencies:

(1) Functional dependencies related to description of entities or relationships. Since an attribute is defined as a function, it maps an entity in an entity set to a single value in a value set (see Figure 2). At level 2, the values of the primary key are used to represent entities. Therefore, nonkey value sets (domains) are functionally dependent on primary-key value sets (for example, in Figures 6 and 7, NO-OF-YEARS is functionally dependent on EMPLOYEE-NO). Since a relation may have several keys, the nonkey value sets will functionally depend on any key value set. The key value sets will be mutually functionally dependent on each other. Similarly, in a relationship relation the nonkey value sets will be functionally dependent on the prime-key value sets (for example, in Figure 8, PERCENTAGE is functionally dependent on EMPLOYEE-NO and PROJECT-NO).

(2) Functional dependencies related to entities in a relationship. Note that in Figure 11 we identify the types of mappings ($1:n$, $m:n$, etc.) for relationship sets. For example, PROJECT-MANAGER is a $1:n$ mapping. Let us assume that PROJECT-NO is the primary key in the entity relation PROJECT. In the relationship relation PROJECT-MANAGER, the value set EMPLOYEE-NO will be functionally dependent on the value set PROJECT-NO (i.e. each project has only one manager).

The distinction between level 1 (Figure 2) and level 2 (Figures 6 and 7) and the separation of entity relation (Figure 7) from relationship relation (Figure 8) clarifies the semantics of functional dependencies among data.

4.1.3 3NF Relations Versus Entity-Relationship Relations. From the definition of "relation," any grouping of domains can be considered to be a relation. To avoid undesirable properties in maintaining relations, a normalization process is proposed to transform arbitrary relations into the first normal form, then into the second normal form, and finally into the third normal form (3NF) [9, 11]. We shall show that the entity and relationship relations in the entity-relationship model are similar to 3NF relations but with clearer semantics and without using the transformation operation.

Let us use a simplified version of an example of normalization described in [9]. The following three relations are in first normal form (that is, there is no domain whose elements are themselves relations):

```
EMPLOYEE (EMPLOYEE-NO)
PART (PART-NO, PART-DESCRIPTION, QUANTITY-ON-HAND)
PART-PROJECT (PART-NO, PROJECT-NO, PROJECT-DESCRIPTION,
               PROJECT-MANAGER-NO, QUANTITY-COMMITTED).
```

Note that the domain PROJECT-MANAGER-NO actually contains the EMPLOYEE-NO of the project manager. In the relations above, primary keys are underlined.

Certain rules are applied to transform the relations above into third normal form:

```
EMPLOYEE' (EMPLOYEE-NO)
PART' (PART-NO, PART-DESCRIPTION, QUANTITY-ON-HAND)
```